

IT314AB Capture The Flag 3

Capture the Flag 3: Data-Driven UI

Goal: Separate application data from the UI and handle multiple pieces of data in a usable interface.

Instructor Note

These activities require your own problem-solving and research skills.

DO NOT use AI to generate the code for you.

You may use AI to:

Research what a Dart/Flutter feature does

Understand syntax

Explain an error (not fix your error)

Clarify documentation

You may NOT ask AI to generate the solution/code for a flag.

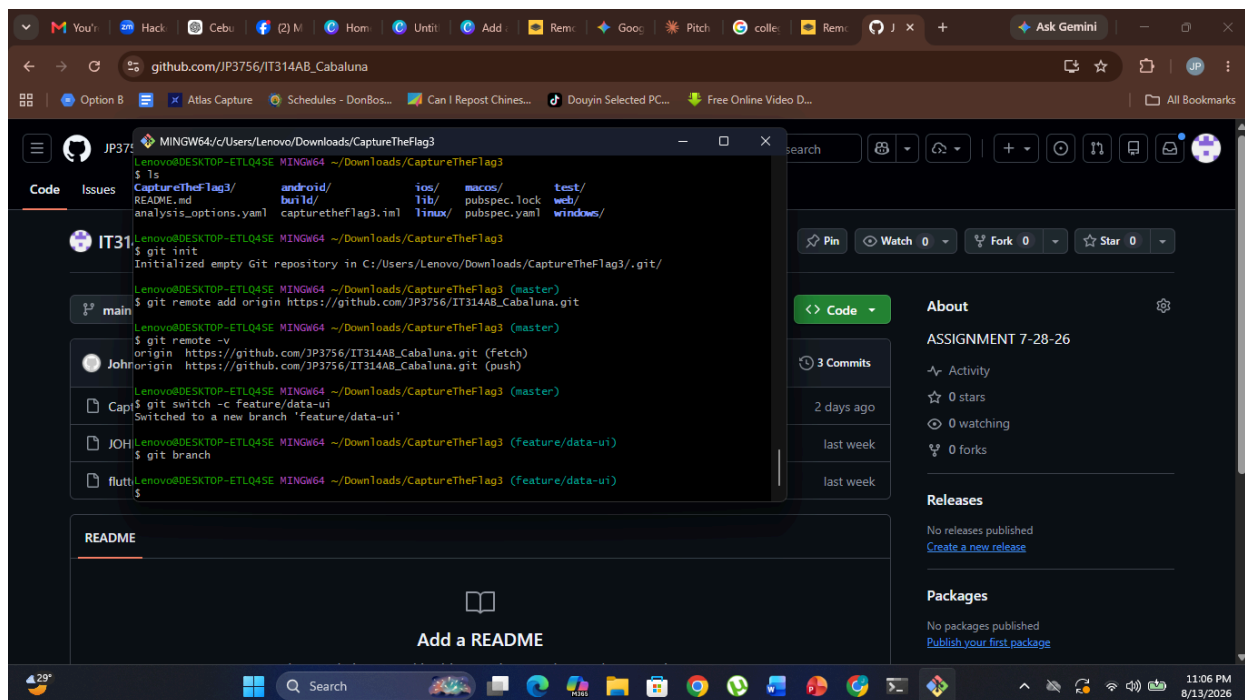
Flag 0 — Create Your Feature Branch

Objective: Create a feature branch before modifying your application.

Create:

feature/data-ui

Screenshot:



Terminal showing your current branch

Flag 1 — Identify the Data

Objective: Identify the appropriate Dart data type for the information in your profile.

Research basic Dart data types and identify at least four different types that could be used in your application.



Examples:

String

int

double

bool

Create a table identifying the information and its appropriate data type.

Screenshot:

Your identified data

The corresponding data types

 Flag 2 — Create Your Data

Objective: Create appropriately typed variables for your profile.

Create variables for at least:

Name

Course & Section

Age

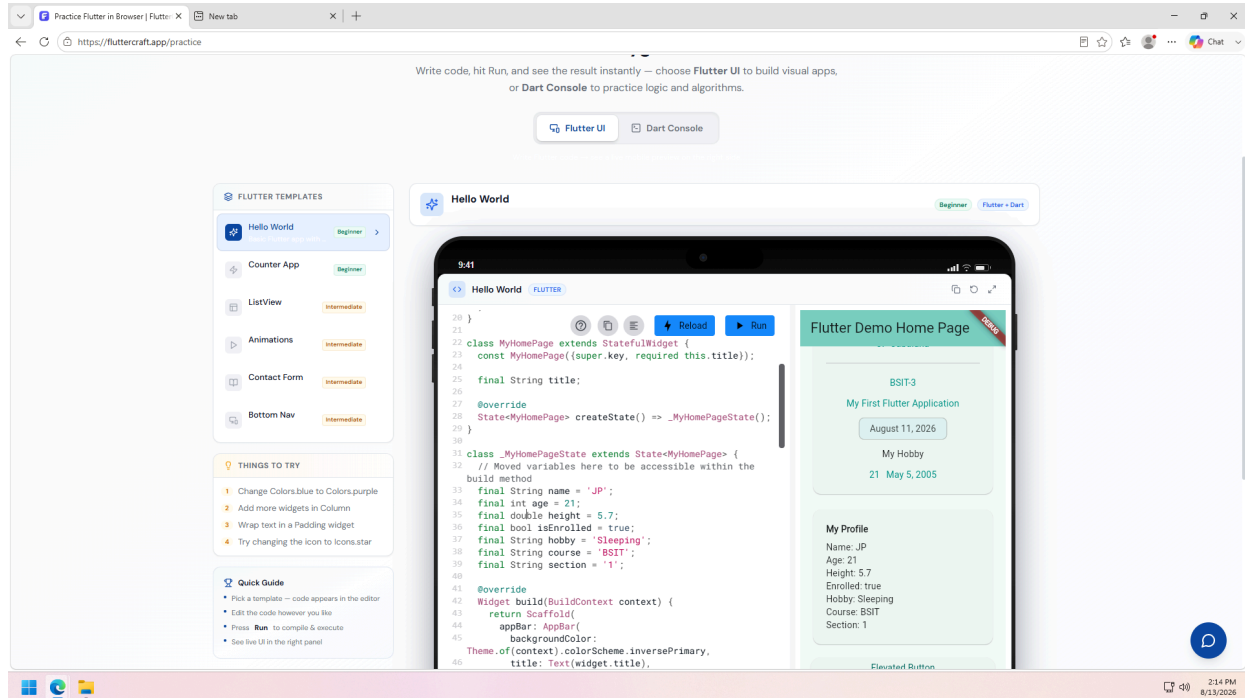
Hobby

Height

Student status

Your data should exist separately from your UI widgets.

Screenshot:



Your variables

Their data types

Application still running

🚩 Flag 3 — Connect Data to UI

Objective: Replace your hardcoded information with your variables.

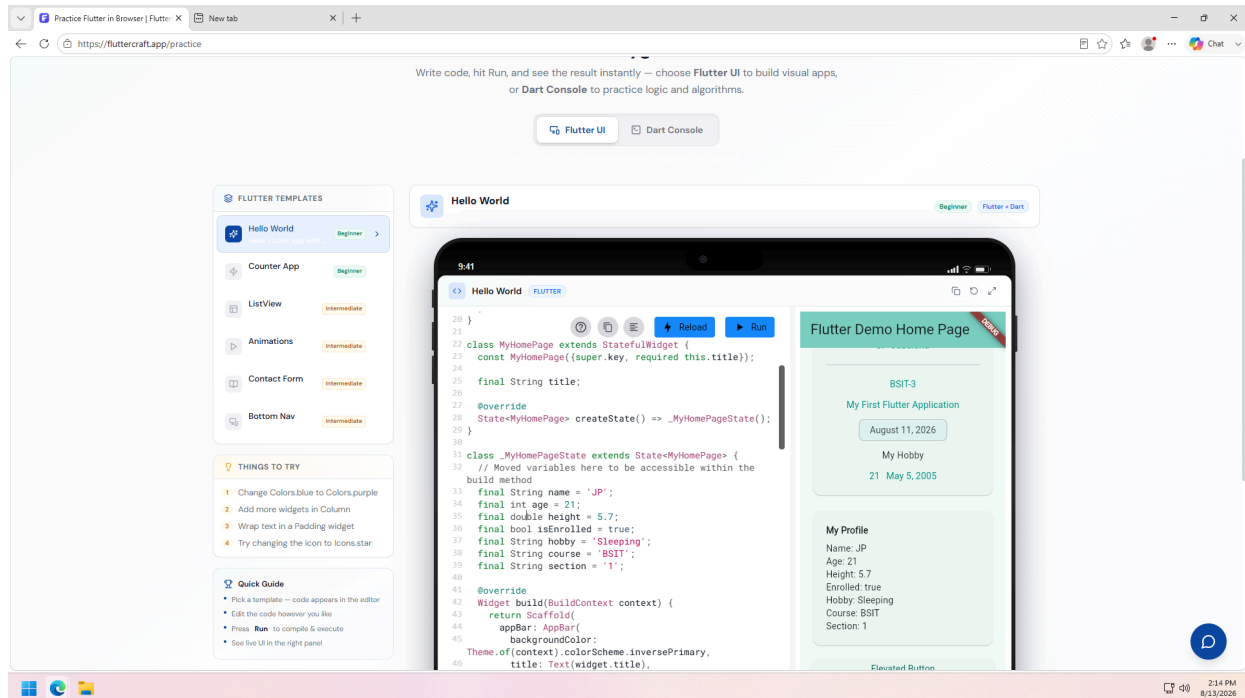
Your profile should now display information using the variables you created.

Test:

Change the value of one of your variables.

The UI should automatically display the new value.

Screenshot:



Variables

Widgets displaying the variables

Updated application

🚩 Flag 4 — Make the Image Data-Driven

Objective: Make your profile image part of your data.

Create a variable containing the path to your profile image.

Use the variable when displaying the image.

Then change the variable to another registered image.

The image displayed by the application should change.

Screenshot:

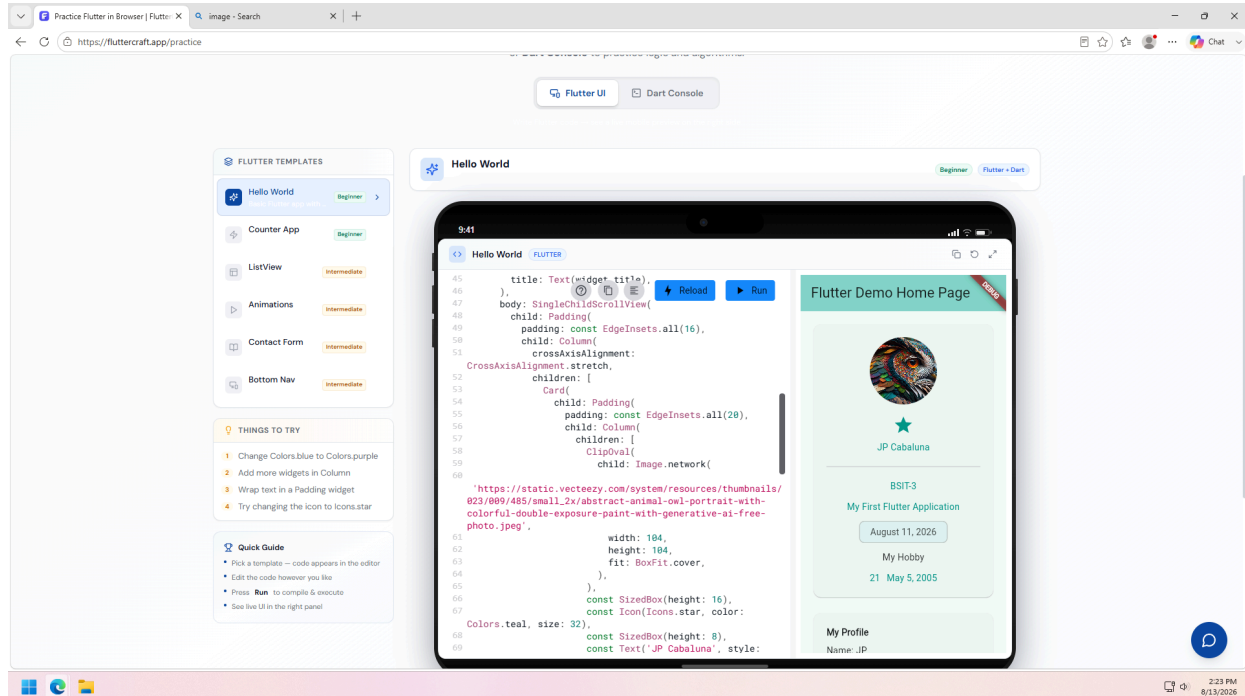


Image variable

Image widget using the variable

Updated application

🚩 Flag 5 — Five Profiles, One Design

Objective: Create at least five different profile cards using the same visual design.

Each profile must contain at least:

Image

Name

Course & Section

Age

Hobby

The five profiles must contain different information.

Important

All five profiles should use the same visual design.

You are demonstrating that the same UI can represent different data.

Screenshot:

Flutter Demo Home Page

DEBUG



My Profile

Name: JP

Age: 21

Height: 5.7

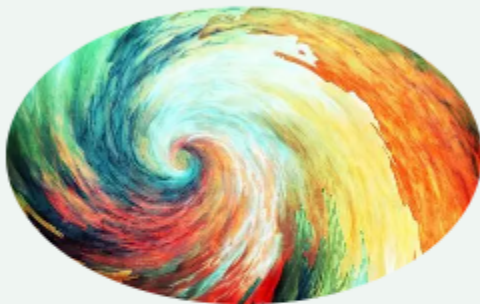
Enrolled: true

Hobby: Sleeping

Course: BSIT

Flutter Demo Home Page

DEBUG



My Profile

Name: PJ

Age: 1

Height: 7.11

Enrolled: YES



My Profile

Name: LIAM
Age: 1
Height: 7.11
Enrolled: NO
Hobby: THINKING
Course: PHILOSOPHY
Section: 5



My Profile

Name: SPIKEY
Age: 1
Height: 7.11
Enrolled: YES
Hobby: KUNG FU
Course: KARATE
Section: 5



My Profile

Name: Jess
Age: 1
Height: 7.11
Enrolled: no
Hobby: eating bugs
Course: coding
Section: 10

Your 5 different profile data

Flag 6 — THE HIDDEN PROBLEM

You didn't see your UI?

Objective: Make all five profiles accessible to the user.

You just added five profile cards.

Now run your application.

Something is wrong.

Look carefully at your emulator.

Can you see all five profiles?

Maybe you can see the first one...

Maybe part of the second...

But where are the rest?

That's right.

You can't see them.

Your UI is larger than the available screen.

The profiles exist.

The data exists.

The widgets exist.

But the user can't access everything.

Your Mission:

Figure out how Flutter allows users to scroll through content that is larger than the available screen.

Research Flutter's scrolling widgets and determine which one is appropriate for your current layout.

You must make it possible to scroll through all five profiles.

Requirement:

The user must be able to:

Scroll → see Profile 1 → Profile 2 → Profile 3 → Profile 4 → Profile 5

without changing the size of the screen.

Screenshot:

```

1 import 'package:flutter/material.dart';
2
3 void main() {
4   runApp(const MyApp());
5 }
6
7 class MyApp extends StatelessWidget {
8   const MyApp({super.key});
9
10  @override
11  Widget build(BuildContext context) {
12    return MaterialApp(
13      title: 'Flutter Demo',
14      theme: ThemeData(
15        colorScheme: ColorScheme.fromSeed(seedColor: Colors.teal),
16      ),
17      home: SingleChildScrollView(
18        child: const MyHomePage(title: 'Flutter Demo Home Page'),
19      ),
20    );
21  }
22 }
23
24 class MyHomePage extends StatefulWidget {

```

The scrolling widget in your code

Top portion of the application

The rest of the portion of the application showing the later profiles

🚩 Flag 7 — Survive the Database

Objective: Prepare your UI for incomplete data.

Now imagine your profiles aren't manually created.

Someday, the data will come from a database.

And real data isn't always perfect.

Some information may be:

Missing

Empty

null

Not provided

Create at least three profiles with missing information.

For example:

Profile A

Name: Maria

Course: BSIT

Hobby: missing

Profile B

Name: Juan

Course: missing

Hobby: Gaming

Profile C

Name: missing

Course: MMA

Hobby: Drawing

Your application must not crash when information is missing.

Instead, provide an appropriate fallback.

Examples:

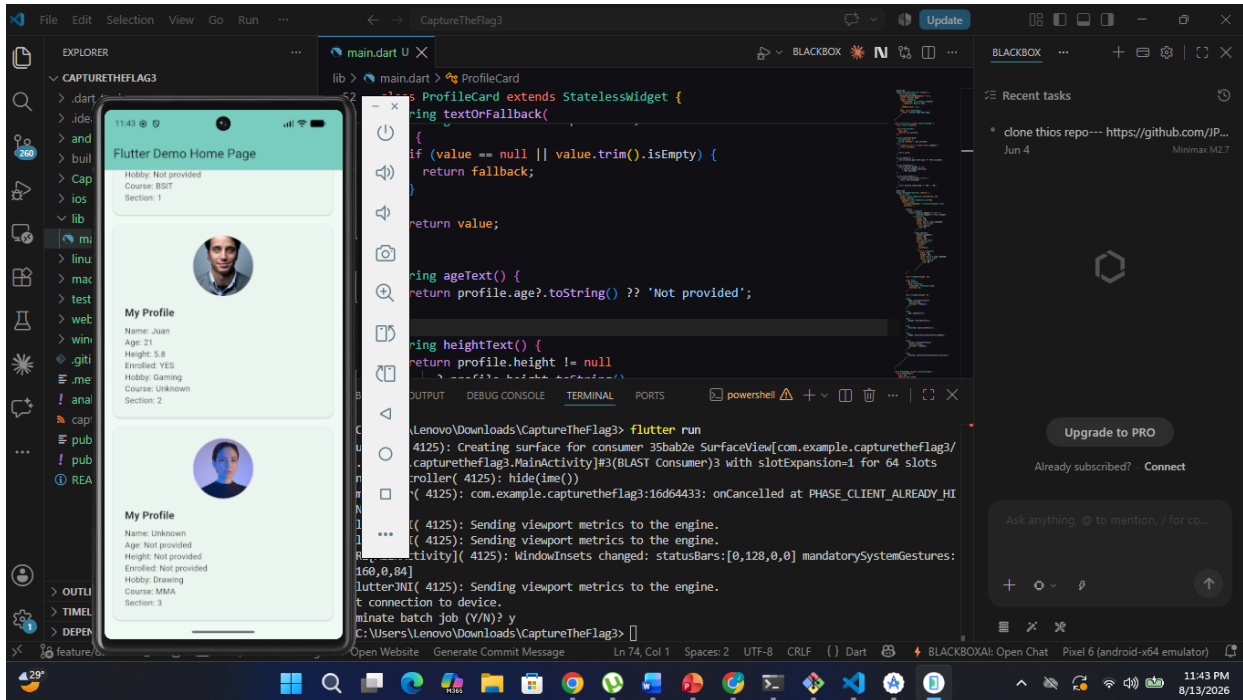
Hobby: Not provided

Course: Unknown

Challenge

Your placeholders should look intentional and clean.

Screenshot:



Incomplete profile data

Application displaying fallback values

Application still functioning correctly

🚩 Flag 8 — Save Your Progress

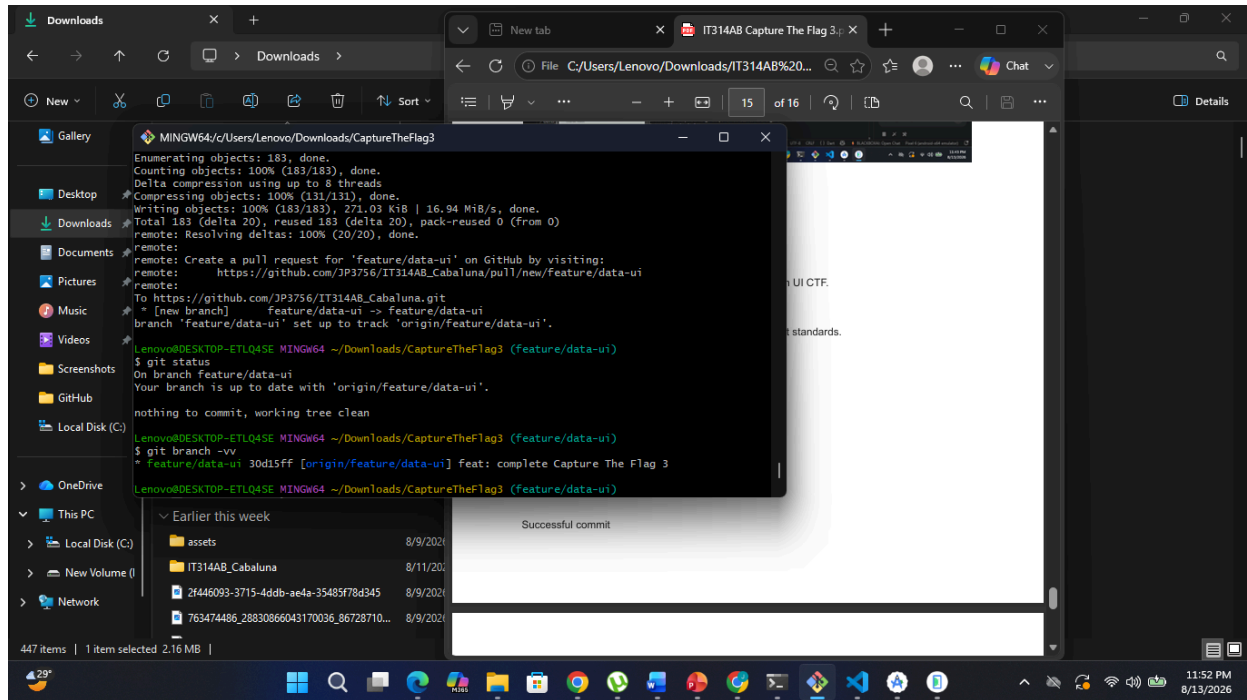
Objective: Commit and push your completed Data-Driven UI CTF.

Use an appropriate commit message following proper Git standards.

Branch:

feature/data-ui

Screenshot:



Successful commit

Current branch

Terminal showing the commit